Automatic for the People and Derivatives in Your Pocket

MOD300: Mandatory project 1

Deadline: 6. September 2026 (23:59)

Aksel Hiorth

Aug 24, 2026

Learning objectives. By completing this project, the student will:

- Get experience in structuring and writing a report.
- Write reusable functions and classes in Python.
- Explore numerical round-off and truncation errors.
- Be introduced to useful Python libraries for scientific computing.
- Collect and analyze real, self-measured data using Python.

Abstract

Because computers have finite memory, numerical errors must always be taken into account when doing calculations, especially when working with floating-point numbers [1]. In the first parts of this project we investigate round-off errors and truncation errors using the Python programming language, and we discuss how different implementation strategies affect code efficiency and code clarity. You will use object-oriented programming to encapsulate data and operations, with automatic differentiation as a compact, specialized example [4]. Automatic differentiation is a cornerstone in training neural networks and in many commercial simulation codes.

At the end you will use the free smartphone app phyphox [3], or a bundled example recording, and apply the very same numerical-differentiation tools you built earlier in the project to real sensor data. Remember to take a look in the Appendix for some tips, and read the guidelines for project submission at the end!

1 Exercise 1: Finite-precision arithmetic

Part 1. Run the following code snippet:

```
import sys
sys.float_info
```

- Pick three numbers, e.g. `max`, `min`, and `epsilon`, and *explain* why they have these values.

Hint: Read the lecture material on the IEEE Standard for floating-point arithmetic.

Part 2. In Python, typing `0.1+0.2` does not (typically) produce the same output as `0.3`. *But,* `0.125+0.25` is *always* equal to `0.375`

- Why? Produce your own examples where precision errors appear and where they do not.

Part 3.

- Should you use the `==`-operator to test whether two floating-point numbers are equal?
- Why / why not? Can you think of alternative ways to do floating-point number comparison?

2 Exercise 2: Get up to speed with NumPy

The purpose of this exercise is to learn a little bit about NumPy, which is an incredibly useful Python library. A major reason for its popularity is efficiency: doing computations with NumPy arrays (objects of the type `ndarray`) instead of using native Python lists (vanilla Python) can, by itself, speed up a program by several orders of magnitude! The mechanism for speed-up is vectorized computation.

Vectorized functions.

Using NumPy arrays allows you to create vectorized functions; functions that operate on a whole array at once, rather than looping over the elements one-by-one inside a custom written loop.

The way vectorization works behind the scenes is still via loops (optimized, pre-compiled C code), but as Python programmer you do not need to worry about the details.

Part 1. The following code block gives an example of a vectorized function:

```
x = np.linspace(0, 1, 10)
np.exp(x)  # Apply f(t)=exp(t) to each element in the array x.
np.exp(-x)  # Apply the function f(t)=exp(-t) to each element of x.
```

Notice the usage of `np.exp` instead of using the exponential function provided in the built-in math library; this is an example of a universal function.

- Create a native Python list of the same size as `x` and holding the same values, e.g. with `x_list = list(x)`. Run

```
np.exp(x_list)
np.exp(-x_list)
```

- Explain why the first call works while the second one fails before NumPy gets a chance to operate on the list.
- How would you generally evaluate a function on all elements of a native Python list? (as opposed to a NumPy array)

Part 2. As already hinted at, the NumPy library comes with a plethora of useful features and functions. The code snippets below show some examples:

```
np.zeros(20)
```

```
np.ones(20)
```

```
np.linspace(0, 10, 11)
```

```
np.linspace(0, 10, 11, endpoint=False)
```

```
vector = np.arange(5) + 1
2*vector
```

- Explain what each line of code does.
- How would you produce the same output using native Python lists?

Part 3. Frequently you will want to extract a subset of values from an array based on some kind of criterion. For example, you might want to count the number of non-zero numbers, or identify all values exceeding a certain threshold. With NumPy, such tasks are easily achieved using boolean masking, e.g.:

```
array_of_numbers = np.array([4, 8, 15, 16, 23, 42,0,5])
nnz = np.count_nonzero(array_of_numbers)
print(f'There are {nnz} non-zero numbers in the array.')
is_even = (array_of_numbers % 2 == 0)
is_greater_than_17 = (array_of_numbers > 17)
is_even_and_greater_than_17 = is_even & is_greater_than_17
```

However, neither of the following code lines will execute:

```
is_even_and_greater_than_17 = is_even and is_greater_than_17
print(array_of_numbers % 2 == 0 & array_of_numbers > 17)
```

- Explain why this code fails.
- Use `np.logical_and` to make the code work

Part 4. The function np.where can also be used to select elements from an array.

- Explain the output of the following two lines of code:

```
np.where(array_of_numbers > 17)[0]
```

```
np.where(array_of_numbers > 17, 1, 0)
```

3 Exercise 3: Build finite-difference tools

In scientific computing we often need the derivative of either a mathematical function or a sequence of measured values. In this exercise you will build finite-difference functions that handle an array of values. You will first test them on a function with a known derivative and later reuse the *same code* on your own phone data.

We use a damped oscillation as our test case:

$$\theta(t) = Ae^{-\gamma t}\cos(\omega_0 t), \qquad A = 0.6, \quad \gamma = 0.05, \quad \omega_0 = \pi. \tag{1}$$

Here θ may be interpreted as an angle in radians and t as time in seconds. A vectorized Python implementation is

```
def damped_angle(t, A=0.6, gamma=0.05, omega0=np.pi):
    return A*np.exp(-gamma*t)*np.cos(omega0*t)
```

It works for both a single number and a NumPy array.

Build small functions and reuse them.

The two finite-difference functions you write below operate only on sampled values and a spacing. A separate wrapper will adapt them to arbitrary Python functions. Keeping these jobs separate makes the code easier to test and is what allows you to reuse the numerical core in Exercise 5.

Part 1. The analytical angular velocity is

$$\omega(t) = \theta'(t) = -Ae^{-\gamma t}\left[\gamma\cos(\omega_0 t) + \omega_0\sin(\omega_0 t)\right]. \tag{2}$$

- Implement equation (2) as a Python function named `damped_velocity`.
- Plot $\theta(t)$ and $\omega(t)$ for $0 \le t \le 10$ s in two panels. Label the axes and include the parameter values in your figure caption.

Part 2. Suppose `y` contains uniformly spaced samples with spacing h. The forward and central differences are

$$D_{\mathrm{f}} y_i = \frac{y_{i+1} - y_i}{h}, \tag{3}$$

$$D_{\mathrm{c}} y_i = \frac{y_{i+1} - y_{i-1}}{2h}. \tag{4}$$

- Write the functions

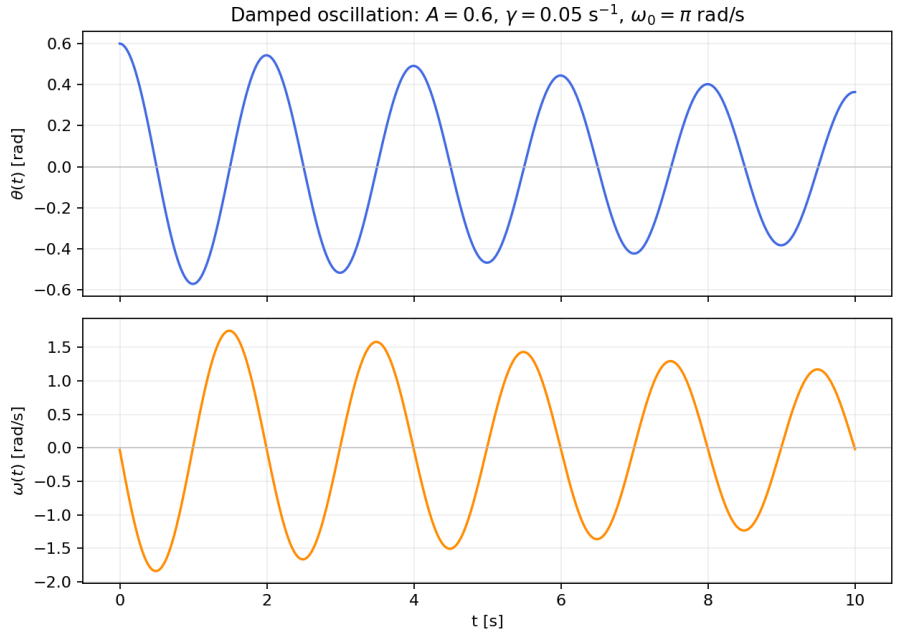

Figure 1: The damped angle and its analytical angular velocity for the default parameters.

```
def forward_diff(y, h):
    # your code

def central_diff(y, h):
    # your code
```

Do not use a Python loop. `forward_diff(y, h)` must return an array of length `len(y)-1` and `central_diff(y, h)` an array of length `len(y)-2`. Test both functions on a linear function, whose derivative is constant, and a quadratic function, whose derivative is known.

Part 3. Python functions are first-class objects: they can be passed as arguments to other functions. Build a wrapper with the following interface:

```
def derivative_at(f, x, h, method, *args, **kwargs):
    # evaluate f at the two or three points needed by method,
    # then call forward_diff or central_diff
```

The argument `method` must be either your `forward_diff` or `central_diff` function. Extra positional and keyword arguments must be passed on to `f`. Appendix A contains a short explanation of `*args` and `**kwargs`.

- Use `derivative_at` to estimate $\omega(0.8)$ with both methods and $h = 10^{-2}$.
- Compare both estimates with `damped_velocity(0.8)`.
- Repeat the calculation with at least one non-default choice of A, γ, or ω_0 to demonstrate that the extra arguments work.

Part 4. Quantify the numerical error at $t = 0.8$:

- Generate logarithmically spaced step sizes from 10^{-16} to 10^{0}.
- For each h, calculate the absolute forward- and central-difference errors.
- Plot both error curves in the same log-log figure.
- Report the best h and the smallest error for each method.
- Explain the large-h truncation-error region, the small-h round-off-error region, and why the central difference normally reaches a smaller error.

Keep `forward_diff` and `central_diff` unchanged after this point: they are part of the code you will reuse in Exercise 5.

4 Exercise 4: Automatic for the people?

The main purpose of this exercise is to practise writing and using Python classes. A class bundles related data and the methods that operate on those data. For example, every Python string is an object with useful methods:

```
x = 'This is a string'
x.upper()
```

The call `x.upper()` returns `'THIS IS A STRING'`. Classes are used throughout engineering software to represent objects such as sensors, materials, geometries, and numerical models. The implementation is written once and can then be reused for many objects.

4.1 A small class first

Here is a class that stores the coordinates of a point together with an operation that belongs naturally to a point:

```
import numpy as np

class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def distance_from_origin(self):
        return np.sqrt(self.x**2 + self.y**2)
```

Part 1.

- Create two `Point` objects and call `distance_from_origin()` for both.
- Add a method `move(self, dx, dy)` that changes the coordinates by `dx` and `dy`. Test it on one of your points.
- Identify the data and the operations that the class keeps together.

4.2 A specialized application: automatic differentiation

Automatic differentiation is a specialized application of classes. We use it here because it gives a compact example of encapsulation: a function value and its derivative are stored together, while the rules of differentiation are implemented once and reused in many calculations. Redefining operators such as `+` and `*` is not necessary for most Python classes, but it is particularly convenient in this example.

Automatic differentiation.

Numerical differentiation approximates a derivative by subtracting nearby values. Automatic differentiation instead carries the value and derivative through every elementary operation by applying the usual calculus rules [4].

For every intermediate quantity we store

$$\begin{pmatrix} f(x) \\ f'(x) \end{pmatrix}. \tag{5}$$

We call this pair a *dual number*. Begin with the following class:

```
class Dual:
    """Store a function value and its derivative."""

    def __init__(self, value, derivative=0.0):
        self.value = value
        self.derivative = derivative

    def __repr__(self):
        return f"Dual(value={self.value}, derivative={self.derivative})"
```

Part 2.

- Create `x = Dual(3, 1)` and `c = Dual(3)`. Print both objects and explain why their derivatives differ.
- Which data are encapsulated in each `Dual` object?

4.3 Addition and subtraction

If u and v depend on x, then

$$(u+v)' = u' + v', \qquad (u-v)' = u' - v'.$$

Create two objects and try to add them:

```
u = Dual(1, 2)
v = Dual(3, 4)
print(u + v)
```

Python initially reports an error because it does not yet know what `+` means for our class. We define the operation once by adding this method inside `Dual`:

```
    def __add__(self, other):
        return Dual(self.value + other.value,
                    self.derivative + other.derivative)
```

In this exercise, arithmetic operations combine two `Dual` objects. Represent an ordinary constant c by `Dual(c)`, because its derivative is zero. Keeping this convention lets us concentrate on the rules of differentiation.

Part 3. Add the worked `__add__` method to your class and check that `u + v` now gives value 4 and derivative 6. Then implement `__sub__` and `__neg__`.

Test your implementation with

```
print(u + v)  # value 4, derivative 6
print(u - v)  # value -2, derivative -2
print(-u)
```

4.4 Multiplication and division

Multiplication and division use the product and quotient rules:

$$(f(x)\cdot g(x))' = f'(x)\cdot g(x) + f(x)\cdot g'(x), \tag{6}$$

$$\left(\frac{f(x)}{g(x)}\right)' = \frac{f'(x)\cdot g(x) - f(x)\cdot g'(x)}{g(x)^2}. \tag{7}$$

Part 4. Implement `__mul__` and `__truediv__` using the rules above.

Test the class on expressions whose analytical derivatives you know:

```
x = 1.2
One = Dual(1.0)       # a constant has derivative 0
X = Dual(x, 1)       # x has derivative 1 with respect to itself
print(X*X*X)         # compare with x**3 and 3*x**2
print(One/(One+X))   # compare with 1/(1+x) and -1/(1+x)**2
```

4.5 Elementary functions

Arithmetic is not enough for our damped oscillation: we also need the chain rule

$$f(g(x))' = f'(g(x)) \cdot g'(x). \tag{8}$$

Part 5. The exponential function provides a pattern:

```
def dual_exp(d):
    value = np.exp(d.value)
    return Dual(value, value*d.derivative)
```

Write `dual_cos(d)` in the same way. It should take a `Dual` and return a new `Dual` containing the cosine and its derivative from the chain rule. Test both functions using `Dual(0.4, 1)`.

Part 6. Use your class to evaluate the damped oscillation from Exercise 3:

```
t = 0.8
T = Dual(t, 1)
A = Dual(0.6)
Gamma = Dual(0.05)
Omega0 = Dual(np.pi)

theta_dual = A*dual_exp(-Gamma*T)*dual_cos(Omega0*T)
print(theta_dual)
```

- Compare the automatic result with the two analytical functions:

```
print(theta_dual.value, damped_angle(t))
print(theta_dual.derivative, damped_velocity(t))
```

- Repeat the calculation at two other values of t. Notice that the class and its differentiation rules do not change when the expression changes.
- Compare the automatic derivative with the forward and central estimates from Exercise 3.

Part 7. In three to five sentences, explain what `Point` and `Dual` encapsulate and where code is reused. Also explain why a class is broadly useful even though automatic differentiation and operator overloading are specialized examples.

5 Exercise 5: Derivatives in your pocket

Make a motion, record it, and run the code you built on your own data. The goal is not a precise physical experiment. We want to see that integrating an angular velocity produces an angle and that differentiating that angle approximately recovers the signal we started with:

$$\omega(t) \xrightarrow{\text{integrate}} \theta(t) \xrightarrow{\text{differentiate}} \widehat{\omega}(t) \approx \omega(t).$$

phyphox (PHYsical PHone eXperiments) is a free app that logs a phone's sensors and exports the data as CSV [3]. Open *Sensor* → *Gyroscope*. Record 10–20 seconds while you safely rock or rotate the phone by hand. You may make a regular oscillation, draw a pattern through the air, or invent another motion.

No suitable phone or recording?

Use the bundled `data/gyroscope_example.csv` and do exactly the same analysis. Using the example data does not reduce the assessment. The interesting part here is the code that you write.

Part 1. Export the recording as CSV and load it with pandas. (NB: The exact column names can vary slightly between app versions.)

- Plot the signed gyroscope x, y, and z channels in the same figure.
- Choose one signed channel with a clearly visible signal and call it `omega`.
- *Do not* use the `Absolute` channel: it has no sign, so clockwise and counterclockwise rotations cannot cancel during integration.

Part 2. The gyroscope measures angular velocity $\omega(t)$ directly, not the angle $\theta(t)$. Extract the timestamp as `t` and use the following implementation of the composite trapezoidal rule:

```
def cumulative_trapezoid(y, t, initial=0.0):
    """Return the cumulative integral of y with the same length as y."""
    areas = 0.5*(y[1:] + y[:-1])*np.diff(t)
    return initial + np.concatenate(([0.0], np.cumsum(areas)))

theta = cumulative_trapezoid(omega, t)
```

Plot the resulting angle $\theta(t)$. You do not need to know its starting value: choosing `initial=0.0` only shifts the entire angle curve vertically.

Part 3. Apply your *unmodified* `forward_diff` and `central_diff` functions from Exercise 3 to `theta`.

- Make an overlay plot containing the original signal and both reconstructed

signals. Zoom in if that makes the comparison clearer.

- Add two or three sentences: did you approximately recover the original signal, and can you

recognize something you did with the phone in the plot?

6 Optional challenge: A derivative in the atmosphere

This part is optional.

The required project ends with your creative phone analysis above. This extension is available if you want to apply the same code to a very different real dataset.

In a deliberately simplified one-box model, atmospheric methane (CH_4) is removed with a fixed lifetime:

$$\frac{d(\Delta M)}{dt} = \frac{E_{CH_4}(t)}{2.75} - \frac{\Delta M}{\tau}, \qquad \tau = 9.1 \text{ years}, \tag{9}$$

Here ΔM [ppb] is the methane concentration anomaly above the pre-industrial value $M_0 = 700$ ppb, $E_{CH_4}(t)$ [Tg/yr] is an effective emission rate under this model, and 1 ppb CH_4 = 2.75 Tg CH_4. Bundled with this project is `data/ch4_annual.dat`: NOAA's global annual mean atmospheric CH_4 record, 1984–2025 [2].

- Rearrange equation (9) to solve for $E_{CH_4}(t)$ in terms of ΔM and its time derivative.
- Estimate $d(\Delta M)/dt$ with your unmodified `central_diff` function and $h = 1$ year.
- Plot the effective emission rate and comment on one visible feature. Be clear that the result follows from the assumed one-box, fixed-lifetime model and is not a full atmospheric inversion.

7 Appendix A: Passing arguments to functions

The wrapper in Exercise 3 must work with functions that have different parameters. Python's `*args` collects extra positional arguments and `**kwargs` collects extra keyword arguments. Both can be forwarded without the wrapper knowing their names:

```
def call_function(f, x, *args, **kwargs):
    return f(x, *args, **kwargs)

# Positional parameters: A, gamma, omega0
call_function(damped_angle, 0.8, 0.6, 0.05, np.pi)

# The same parameters passed by name
call_function(
    damped_angle,
    0.8,
    A=0.6,
    gamma=0.05,
    omega0=np.pi,
)
```

Your `derivative_at` function uses exactly the same forwarding pattern when it evaluates `f` at the points required by the chosen finite-difference method:

```
derivative_at(
    damped_angle,
    0.8,
    1e-3,
    central_diff,
    A=0.6,
    gamma=0.05,
    omega0=np.pi,
)
```

8 Appendix B: Getting set up with phyphox and Python

8.1 Installing phyphox

phyphox is free and available from the app store on your phone:

- iOS: search for "phyphox" in the App Store.
- Android: search for "phyphox" in the Google Play Store.

No account or sign-up is required. Once installed, browse to *Sensor* → *Gyroscope* (the experiment used in Exercise 5), press record, and use the export button (usually a share/export icon) to save your recording as a CSV file.

8.2 Python environment

Unlike previous years, this project does *not* need any exotic packages – a standard scientific-Python environment with `numpy`, `scipy`, `matplotlib`, and `pandas` is all you need. If you do not already have such an environment, you can create one with `conda`:

```
conda create -n project1 python=3.11 numpy scipy matplotlib pandas jupyter -c conda-forge
conda activate project1
```

Remember to select this environment (or its Jupyter kernel) in whichever tool you use to write your notebook (VS Code, Jupyter, Colab, ...).

9 Guidelines for project submission

You should bear the following points in mind when working on the project:

- Write the name of all persons working in the group at the top of the notebook.
- The final project report must start with an abstract, the abstract is a self contained summary of the project and should contain quantitative statements.
- Start your notebook by providing a short introduction in which you outline the nature of the problem(s) to be investigated.
- End your notebook with a brief summary of what you feel you learned from the project (if anything). Also, if you have any general comments or suggestions for what could be improved in future assignments, this is the place to do it.
- All code that you make use of should be present in the notebook, and it should ideally execute without any errors (especially run-time errors). If you are not able to fix everything before the deadline, you should give your best understanding of what is not working, and how you might go about fixing it.
- Avoid duplicating code! If you find yourself copying and pasting a lot of code, it is a strong indication that you should define reuseable functions and/or classes.
- If you use an algorithm that is not fully described in the assignment text, you should try to explain it in your own words. This also applies if the method is described elsewhere in the course material.
- In some cases it may suffice to explain your work via comments in the code itself, but other times you might want to include a more elaborate explanation in terms of, e.g., mathematics and/or pseudocode.

- In general, it is a good habit to comment your code (though it can be overdone).
- When working with approximate solutions to equations, it is very useful to check your results against known exact (analytical) solutions, should they be available.
- It is also a good test of a model implementation to study what happens at known 'edge cases'.
- Any figures you include should be easily understandable. You should label axes appropriately, and depending on the problem, include other legends etc. Also, you should discuss your figures in the main text.
- It is always good if you can reflect a little bit around *why* you see what you see.

References

[1] David Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM computing surveys (CSUR)*, 23(1):5–48, 1991.

[2] NOAA Global Monitoring Laboratory. Trends in atmospheric methane, global monitoring laboratory. https://gml.noaa.gov/ccgg/trends_ch4/, 2025. Annual global mean CH_4 mole fraction. Accessed: 2026-07-30.

[3] Sebastian Staacks, Simon Hutz, Heidrun Heinke, and Christoph Stampfer. Advanced tools for smartphone-based experiments: Phyphox, 2018.

[4] Robert Edwin Wengert. A simple automatic derivative evaluation program. *Communications of the ACM*, 7(8):463–464, 1964.